07年期末考试试卷点评

鲁仲阳

lzhy@fudan.edu.cn



选择(5)

(5) 在 C 语言中，如果在定义变量时没有对变量做显式初始化，自动有初值“机器零”（每个二进制位都为 0）的是_____。

A. char 型的变量　　B. 所有指针类型的变量

C. 各种类型的全局变量　　D. 函数的形参

具有全局生命周期的变量(包括数组)，在定义时如果没有初始化，则初值自动为0，而具有动态生存期的变量未初始化时，初值为一随机数

变量复习(1)

- 变量的存储类型

  C语言中的变量存储类型有4种，分别为auto（自动变量），static（静态变量），extern（外部变量），register（寄存器变量）

变量复习(2)

1.全局变量的存储类型：

- static和extern两种，其中extern是缺省的全局变量存储类型。
- int x,y=5;//默认为外部全局变量
- extern int a=1,b=2;//定义外部全局变量，使用关键字extern后，必须进行初始化！
- static float f1,f2;//静态全局变量extern型全局变量通过外部全局变量说明可以被其他源程序引用，而static星的全局变量只能局限在本源程序文件中使用
- 如果使用了关键字extern，则必须对这个外部全局变量进行出示化，否则编译程序会把它当作外部变量的声明（说明）

变量复习(3)

- 2.局部变量的存储类型：

  C语言种的局部变量的存储类型有如下3种：auto，static，register。其中auto是缺省的局部变量存储类型！

- ```
  void main()
  {
      auto int a;//自动变量
      fioat f;//自动变量
      static int i;//静态变量
      register double d;//寄存器变量，要求编译器使用CPU寄存器来存储变量值
  }
  ```

- C语言函数定义中的形参也是局部变量，形参的默认存储类型是auto，也可以指定形参的存储类型是register，但不允许指定形参的村塾类型为static！
  ```

变量复习(4)

- 变量的生存期

C语言的变量生存期有2种：全程生存期，动态生存期。

- 1.全程生存期
- 指变量在程序运行一开始就被建立，而在程序运行结束后才从内存中删除。
- 具有全程生存期的变量是全局变量和静态变量
- 具有全程生存期的变量存储在程序的静态数据存储区中

变量复习(5)

- 2.动态生存期
  - 指变量在程序运行过程中，因需要使用才建立，而使用结束就被删除
  - 具有动态生存期的变量有：局部自动变量，寄存器变量
  - 具有动态生存期的变量存储在程序的动态数据区中
  - 对于复合语句种定义的auto型的局部变量，每当程序运行至该语句时，就会在动态数据区建立这些局部变量的存储空间，而一旦程序流程离开该复合语句时，该复合语句种的局部变量将被系统从内存种删除

变量复习(6)

- 具有全局生存期的变量(包括数组）在定义时如果没有初始化，则初值自动为0；而具有动态生存期的变量未初始化时，初值为一随机数！
- 具有全局生存期的变量在定义时如有初始化，则初始化在程序刚开始运行建立静态数据区时会被执行一次，以后再也不会执行初始化操作。而动态生存期的变量在定义时如有初始化，则每当程序创建该变量时都会执行初始化。

填空（1）

(1)请用文字解释下列定义中 p 的类型是什么？

A） `int (*p)();`

B） `int (*p)[3];`

C） `int *p[4];`

(1) A) p 是函数指针，所指向的函数具有返回值 int

B) p 是指向整形数组的指针，数组元素个数为 3

C) p 是指针数组，每个数组元素（指针）指向一个整形量

填空（4）

（4）写出下面程序执行后的屏幕显示结果

```
#include <stdio.h>
void main()
{
    int i=0, j=0;
    for(i=1; ;i++)
        if (i-10)
            break;
        else
            ++j;
    printf("j=%d",j);
}
```

（4）_____ j=0 _____

填空（6）

(6)写出下面程序执行后的屏幕显示结果

```
#include <stdio.h>
int a=2,c=4;
void f(int a, int *x){
    int b=10;
    static int c=20;
    b+=a++;
    c+=a+b;
    *x=c+2;
}
void main(){
    f(3+c, &a);
    f(a,&c);
    printf("a=%d,c=%d\n",a,c);
}
```

(6) a=47,c=152

填空（7）

（7）下面的程序执行到最后，打印出来的变量 count 的值是多少？

```
#include <stdio.h>
int hanoi (int n,char A,char C,char B){
    static int i=1;
    if ( n == 1 ){
          printf("%d: move %c  to %c\n",i, A, C);
          i++;
    }
    else {
          hanoi ( n-1, A, B, C );
          printf( "%d: move %c  to %c\n",i, A, C);
          i++;
          hanoi ( n-1, B, C, A );
    }
    return i;
}

   void main(){
       unsigned int count;
       count=hanoi(10,'A','C','B');
       printf("%d", count);
   }
```

(7) 1024

填空（8-9）

（8）~（9） 勒让得多项式的递归定义如下，随后的函数用非递归方法求勒让得多项式 p(n,x)的值，返回求值结果。

$$p(n,x)=\begin{cases}1, & n=0;\\ x, & n=1;\\ ((2n-1).x.p(n-1,x)-(n-1)p(n-2,x))/n, & n>1。\end{cases}$$

```
double p(int n, double x){
        double first, second, third;
        int count;
        if (n == 0)  return 1.0;
        if (n == 1)  return x;
        first  = 1.0;
        second = x;
        for(_________(8)_________;count++)   {
              third  = ((2.0*count-1.0)*x*second
                       - (count-1.0)*first)/count;
              _________(9)_________;
              second = third;
        }
        return  third;
}
```

(8) count = 2; count <= n

(9) first = second

填空（10-12）

（10）~（12）不带辅助表元链表颠倒函数。如需颠倒链表的头指针为 h，调用 reverse(&h) 即可把其颠倒。链表表元结构定义如下：

```
struct  intNode  {   //整数链表表元类型
    int  value ;      //存放整数
    struct  intNode  *next ;    //存放后继表元的指针
};
void reverse (struct intNode **hpt){
    struct intNode *p, *v1, *v2;
    v1 = NULL;  //开始颠倒时，已颠倒部分为空
    v2 = *hpt;     //v2 指向链表的首表元
    while (          (10)          ) {
        p = v2->next;
        v2->next = v1;
             (11)          ;
        v2 = p;
    }
          (12)          ;
}
```

(10) v2 != NULL　　(11) v1 = v2　　(12) *hpt = v1

填空（13-14）

（13）~（14）以下程序的功能是从键盘上输入若干个字符（以回车符作为结束）组成一个字符串存入一个字符数组

```
#include <stdio.h>
void main()
{
    char str[81];
    int i;
    for(i = 0; i < 80; i++)
    {
        str[i] = getchar();
        if (str[i] == '\n')
            _____(13)_____ ;
    }
    _____(14)_____;
}
```

(13) break

(14) str[i] ='\0'

改错(1)

(1)下面的程序要打印0、1、2、3、4这5个数，且每个数各占一行。

```
/*1*/  #include <stdio.h>;
/*2*/  #define PRINT(a) printf("%d", a);printf("\n")
/*3*/  void main()
/*4*/  {
/*5*/      int i;
/*6*/      for(i=0; i<5; )
/*7*/          PRINT(i++);
/*8*/  }
```

(1) 第 1 题第 7 行，改为 {PRINT(i++);}

改错(3)

（3）下面的程序要读入正文文件的内容，统计文件中英文字母的个数，并输出。设程序要统计的正文文件名在程序启动时由输入指定。

```
/*1*/  #include  <stdio.h>
/*2*/  void main(){
/*3*/      FILE *fp;
/*4*/      int count;
/*5*/      char ch;
/*6*/      char fname[40];
/*7*/      printf("输入文件名!\n");
/*8*/      scanf("%s%*c", fname);  //读入文件名和名后的回车符
/*9*/      if ((fp = fopen(fname, 'r')) == EOF){
/*10*/         printf("Can not open %s file.\n", fname);
/*11*/         return 1;  // 程序非正常结束
/*12*/     }
/*13*/
/*14*/     count = 0;
/*15*/     while((ch = fgetc(fp)) != EOF) {
/*16*/         if('a'<ch<='z'||'A'<=ch<='Z')
/*17*/             count++;
/*18*/     }
/*19*/     fclose(fname);
/*20*/     // 输出处理结果
/*21*/     printf("文件%s有英文字母%d个.\n",fname,count);
/*22*/     return 0; // 程序执行正常结束
/*23*/ }
```

(6) 第 3 题第 2 行，改为 `int main()`

(7) 第 3 题第 5 行，改为 `int ch;  // signed char ch;`

(8) 第 3 题第 9 行，改为 `if ((fp = fopen(fname, "r")) == NULL){`

(9) 第 3 题第 16 行，改为 `if('a'<ch && ch<='z'||'A'<=ch && ch<='Z')`

(10) 第 3 题第 19 行，改为 `fclose(fp);`

signed char取值范围是 -128 到 127(有符号位) unsigned char 取值范围是 0 到 255